

EMBEDDED LINUX SYSTEMS

LAB 01 REPORT

CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG LINUX NHÚNG (ARMv7)

Họ và tên	Nguyễn Đăng Quang
MSSV	SE201072
Nhóm	2
Ngày thực hiện	08/10/2026

Target: ARMv7-A Cortex-A9 | QEMU vexpress-a9 | Linux Kernel 5.15

1. Mục tiêu bài lab

Bài lab thực hành toàn bộ chuỗi khởi động của một hệ thống Linux nhúng ARMv7: cấu hình và build Linux kernel bằng cross-compilation, sử dụng U-Boot, tạo root filesystem tối giản bằng BusyBox, đóng gói initramfs và boot hệ thống trên QEMU vexpress-a9.

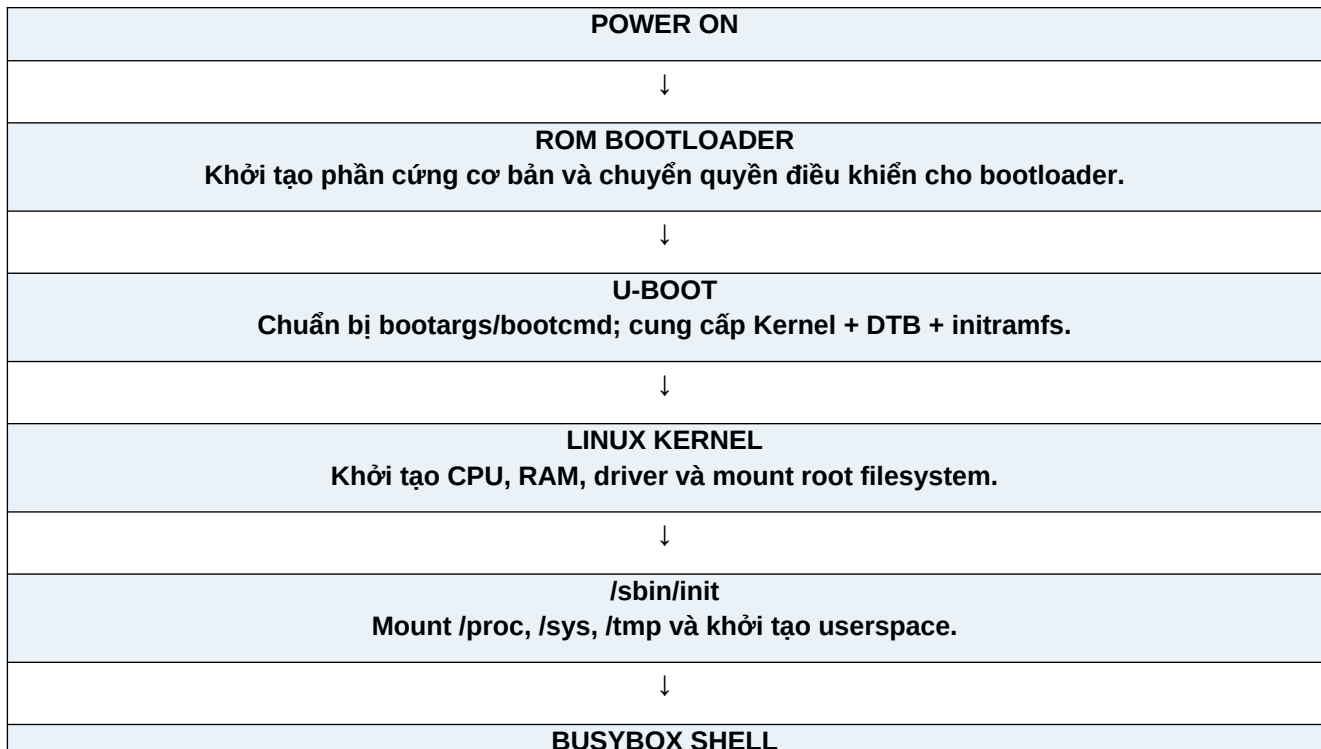
Kết quả cần đạt là kernel ARMv7 có thể khởi động, nhận đúng console PL011, sử dụng initramfs và đi tới BusyBox shell.

2. Môi trường thực hành

Thành phần	Cấu hình sử dụng
Host OS	Ubuntu 24.04 (môi trường thực tế của máy thực hành)
ARM Toolchain	arm-linux-gnueabihf-gcc 13.3.0
QEMU	QEMU emulator 8.2.2
Target	ARMv7-A / Cortex-A9 / vexpress-a9
Kernel	Linux 5.15
U-Boot	2022.04
BusyBox	1.35.0

3. Boot Sequence Diagram

Chuỗi khởi động được mô tả như sau:



EMBEDDED LINUX SYSTEMS — LAB 01 REPORT

Hệ thống sẵn sàng tại prompt / #.		
Bước	Công việc chính	Dữ liệu chuyển tiếp
Power ON / ROM	Khởi tạo phần cứng cơ bản và tìm bootloader.	Quyền điều khiển cho U-Boot.
U-Boot	Thiết lập bootargs/bootcmd, chuẩn bị kernel, DTB và initramfs.	Kernel image, DTB, initramfs và kernel command line.
Linux Kernel	Khởi tạo CPU, memory, driver, mount root filesystem.	Chạy tiến trình init đầu tiên.
/sbin/init	Mount proc/sys/tmp và thực hiện script khởi động.	Khởi tạo userspace.
BusyBox Shell	Cung cấp môi trường lệnh tối giản để kiểm tra hệ thống.	Prompt / # cho người dùng.

4. Log thực hành

4.1. Phần A — Cấu hình và Build Linux Kernel

Kernel được cấu hình cho kiến trúc ARM và board Versatile Express. Các tùy chọn initramfs, PL011 serial console và SMP được kiểm tra trong menuconfig.

```
(13) Temporary per-CPU printk log buffer size (12 => 4KB, 13 => 8KB)
    Scheduler features ----
[*] Control Group support --->
-* Namespaces support --->
[ ] Checkpoint/restore support
[ ] Automatic process group scheduling
[ ] Enable deprecated sysfs features to support old userspace tools
[ ] Kernel->user space relay support (formerly relays)
[*] Initial RAM filesystem and RAM disk (initramfs/initrd) support
() Initramfs source file(s)
[*] Support initial ramdisk/ramfs compressed using gzip
[*] Support initial ramdisk/ramfs compressed using bzip2
[*] Support initial ramdisk/ramfs compressed using LZMA
[*] Support initial ramdisk/ramfs compressed using XZ
```

Hình 1. Initramfs/initrd support đã được bật trong Kernel menuconfig.

```
< > 8250/16550 and compatible serial support
*** Non-8250 serial port support ***
< > ARM AMBA PL010 serial port support
<+> ARM AMBA PL011 serial port support
[*] Support for console on AMBA serial port
[ ] Early console using ARM semihosting
```

Hình 2. ARM AMBA PL011 serial port và console support đã được bật.

```
[*] Symmetric Multi-Processing
[*] Allow booting SMP kernel on uniprocessor systems
[*] Support cpu topology definition
[ ] Multi-core scheduler support
```

Hình 3. Symmetric Multi-Processing (SMP) đã được bật.

Sau khi build, file zImage và Device Tree Blob vexpress-v2p-ca9.dtb được kiểm tra. Lệnh file xác nhận zImage là Linux kernel ARM boot executable.

```
nguyendangquang@nguyendangquang-Bellator:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/zImage
ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 nguyendangquang nguyendangquang 4.9M Sep 29 18:44 arch/arm/boot/zImage
-rw-rw-r-- 1 nguyendangquang nguyendangquang 14K Sep 29 18:56 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
nguyendangquang@nguyendangquang-Bellator:~/embedded_lab1/kernel/linux-5.15$ file arch/arm/boot/zImage
arch/arm/boot/zImage: Linux kernel ARM boot executable zImage (little-endian)
nguyendangquang@nguyendangquang-Bellator:~/embedded_lab1/kernel/linux-5.15$ cp arch/arm/boot/zImage ~/embedded_lab1/output/
cp arch/arm/boot/dts/vexpress-v2p-ca9.dtb ~/embedded_lab1/output/

ls -lh ~/embedded_lab1/output/
total 10M
-rw-rw-r-- 1 nguyendangquang nguyendangquang 1.1M Sep 26 22:58 initramfs.cpio.gz
-rwxrwxr-x 1 nguyendangquang nguyendangquang 4.2M Sep 26 22:39 u-boot
-rw-rw-r-- 1 nguyendangquang nguyendangquang 14K Oct 8 01:27 vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 nguyendangquang nguyendangquang 4.9M Oct 8 01:27 zImage
nguyendangquang@nguyendangquang-Bellator:~/embedded_lab1/kernel/linux-5.15$
```

Hình 4. Kiểm tra zImage, DTB và sao chép các output sang thư mục output.

Nhận xét: zImage có kích thước khoảng 4.9 MB và DTB khoảng 14 KB. Lệnh file xác nhận zImage là Linux kernel ARM boot executable. Ảnh log build đầy đủ từ lần build ban đầu không còn được lưu, vì

vậy báo cáo sử dụng kết quả zImage/DTB thực tế trong thư mục output làm bằng chứng cho sản phẩm sau build.

4.2. Phần B — Cấu hình và sử dụng U-Boot

U-Boot được khởi động trên QEMU vexpress-a9. Các biến môi trường được kiểm tra bằng printenv, sau đó bootargs và bootcmd được thiết lập cho target.

```
devicetree = enable
arch_number = 0x00000000
tlb_addr = 0x7fff0000
irq_sp = 0x7fb2ae00
ip_start = 0x7fb2ae00
[Early malloc usage: 370 / 400
=> printenv
archname
baudrate=38400
board=vexpress
board_name=vexpress
boot_a_script=load $devtype $devnum:$distro_bootpart $scriptaddr $(prefix)$script; source $scriptaddr
boot_efi_binary=load $devtype $devnum:$distro_bootpart $(kernel_addr_r) efi/boot/bootarm.efi; if fdt addr $(fdt_addr_r); then bootefi $(kernel_addr_r) $(fdt_addr_r); else bootefi $(kernel_addr_r) $(fdtcontroladdr); fi
boot_efi_bootmgr=if fdt addr $(fdt_addr_r); then bootefi bootmgr $(fdt_addr_r); else bootefi bootmgr=if
boot_extlinux=syboot $devtype $devnum:$distro_bootpart any $(scriptaddr) $(prefix)$boot_syllinux_conf
boot_prefixes=/ /boot/
boot_script_dhcp=boot.scr.ublg
boot_script=boot.scr.ublg boot.scr
boot_syllinux_conf=extlinux/extlinux.conf
boot_targets=mnc1 mem0 pxe dhcp
bootargs=root=/dev/sdai rw console=ttyAMA0,38400n8 mem=1024M mtdparts=arnflash:1M0x800000(uboot),7M0x1000000(kernel),24M0x2000000(initrd) mnc1.fmax=190000 devtmpfs.mount=0 vmlloc=256M
bootcmd=run distro_bootcmd; run bootflash
bootcmd_dhcp=devtype=dhcp; if dhcp $(scriptaddr) $(boot_script_dhcp); then source $(scriptaddr); fi;setenv efi_fdtfile $(fdtfile); if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; setenv efi_old_vcl ${boot_vcl};setenv efi_old_arch $(boot_arch);setenv bootp_vcl ${fdtcontroladdr};setenv bootp_arch ${kernel_addr_r}; then tftpboot $(fdt_addr_r) dtb/${efi_fdtfile}; if fdt addr $(fdt_addr_r); then bootefi $(kernel_addr_r) $(fdt_addr_r); else bootefi $(kernel_addr_r) $(fdtcontroladdr); fi; fi;setenv bootp_vcl ${efi_old_vcl};setenv bootp_arch $(efi_old_arch);setenv efi_fdtfile;setenv efi_old_arch;setenv efi_old_vcl;
bootcmd_mnc1=devnum=0; run mnc_boot
bootcmd_mnc1=devnum=0; run mnc_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=2
bootfile=boot.scr.ublg
bootflash=run flashargs; cp $(randisk_addr) $(randisk_addr_r) $(maxrandisk); bootn $(kernel_addr) $(randisk_addr_r)
console=ttyAMA0,38400n8
cpurnv7
distro_bootcmd=for target in $(boot_targets); do run bootcmd_${target}; done
dram=1024M
efi_dtb_prefixes=/dtb/ fdtb/current/
ethact=ethernet0,02000000
ethaddr=52:54:00:12:34:56
fdt_addr_r=0x00000000
fdtcontroladdr=7ffdb0
fdtfile=vexpress-v2p-ca9.dtb
flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mnc1.fmax=190000 devtmpfs.mount=0 vmlloc=256M
kernel_addr_r=0x01000000
load_efi_dtb=load $devtype $devnum:$distro_bootpart $(prefix)$efi_fdtfile
loadaddr=0x90000000
mnc_boot=if mnc dev $devnum; then devtype=mnc; run scan_dev_for_boot_part; fi
mtd=arnflash:1M0x800000(uboot),7M0x1000000(kernel),24M0x2000000(initrd)
root=/dev/sdai rw
scan_dev_for_boot=echo Scanning $devtype $devnum:$distro_bootpart....; for prefix in $(boot_prefixes); do run scan_dev_for_extlinux; run scan_dev_for_scripts; done;run scan_dev_for_efi
scan_dev_for_boot_part=for devtype $devnum $bootable devplist; env exists devplist || setenv devplist 1; for distro_bootpart in $(devplist); do if fstype $devtype $devnum:$distro_bootpart bootfstype; then run scan_dev_for_boot; fi; done; setenv devplist
scan_dev_for_efi=setenv efi_fdtfile $(fdtfile); if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; for prefix in $(efi_dtb_prefixes); do if test -e $devtype $devnum:$distro_bootpart $(prefix)$efi_fdtfile; then run load_efi_dtb; fi;done;run boot_efi_bootmgr; if test -e $devtype $devnum:$distro_bootpart efi/boot/bootarm.efi; then echo Found EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile
scan_dev_for_extlinux=if test -e $devtype $devnum:$distro_bootpart $(prefix)$boot_syllinux_conf; then echo Found $(prefix)$boot_syllinux_conf; run boot_extlinux; echo SCRIPT FAILED: continuing...; fi
scan_dev_for_scripts=for script in $(boot_scripts); do if test -e $devtype $devnum:$distro_bootpart $(prefix)$script; then echo Found U-Boot script $(prefix)$script; run boot_a_script; echo SCRIPT FAILED: continuing...; fi; done
stdn=serial
stdout=serial
stderr=serial
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env set bootubivol boot; if ubi part $(bootubipart) && ubifs mount ubi$(devnum):$(bootubivol); then devtype=ubi; run scan_dev_for_boot; fi
vendor=armltd
Environment size: 4438/262140 bytes
=> []
```

Hình 5. Kết quả printenv trong U-Boot.

```
stdn=serial
stdout=serial
stderr=serial
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env set bootubivol boot; if ubi part $(bootubipart) && ubifs mount ubi$(devnum):$(bootubivol); then devtype=ubi; run scan_dev_for_boot; fi
vendor=armltd
Environment size: 4438/262140 bytes
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=> []
```

Hình 6. Thiết lập và kiểm tra bootargs.

bootargs sử dụng console=ttyAMA0,115200, root=/dev/ram0, chế độ rw, init=/sbin/init và mem=512M.

```
vendor=armltd
Environment size: 4438/262140 bytes
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=> printenv bootcmd
bootcmd=bootz 0x60100000 0x64000000 0x60000000
=> []
```

Hình 7. Thiết lập bootcmd với địa chỉ kernel, initramfs và DTB.

bootcmd sử dụng bootz 0x60100000 0x64000000 0x60000000. Các địa chỉ này nằm trong vùng RAM của vexpress-a9 bắt đầu từ 0x60000000.

4.3. Kiểm tra trường hợp chưa có root filesystem

Kernel được boot mà không cung cấp initramfs. Kernel có thể khởi tạo nhưng không có root filesystem để mount, do đó dừng với Kernel Panic.

```
[<00047490>] [panic] from [<00001740>] (mount_block_root+0x10/0x274)
[<00001740>] (mount_block_root) from [<00001000>] (prepare_namespace+0x10/0x10c)
[<00001000>] (prepare_namespace) from [<0000f09c>] (kernel_init+0x10/0x124)
[<0000f09c>] (kernel_init) from [<00100130>] (.ret_from_fork+0x14/0x24)
Exception stack(0x810a77d0 to 0x810a77f0)
7f0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fd: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
-- [ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ] ---
```

EMBEDDED LINUX SYSTEMS — LAB 01 REPORT

Hình 8. Kernel Panic khi không có root filesystem.

Thông báo “VFS: Unable to mount root fs” cho thấy root filesystem là thành phần bắt buộc trước khi kernel có thể chuyển sang userspace.

4.4. Phần C — Root Filesystem, initramfs và boot hoàn chỉnh

Root filesystem tối giản được tạo với các thư mục bin,/sbin, dev, proc, sys, tmp, etc, usr và file init. Sau đó filesystem được đóng gói thành initramfs.cpio.gz.

```
nguyendangquang@nguyendangquang-Bellator: $ cd ~/embedded_lab1/rootfs/initramfs
ls -lh
ls -lh ~/embedded_lab1/output/initramfs.cpio.gz
total 40K
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:55 bin
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:58 dev
drwxrwxr-x 3 nguyendangquang nguyendangquang 4.0K Sep 26 22:57 etc
-rwxrwxr-x 1 nguyendangquang nguyendangquang 76 Sep 26 22:55 init
drwxrwxr-x 3 nguyendangquang nguyendangquang 4.0K Sep 26 22:54 lib
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:54 proc
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:55 sbin
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:54 sys
drwxrwxr-x 2 nguyendangquang nguyendangquang 4.0K Sep 26 22:54 tmp
drwxrwxr-x 4 nguyendangquang nguyendangquang 4.0K Sep 26 22:54 usr
-rw-rw-r-- 1 nguyendangquang nguyendangquang 1.1K Sep 26 22:58 /home/nguyendangquang/embedded_lab1/output/initramfs.cpio.gz
nguyendangquang@nguyendangquang-Bellator: ~/embedded_lab1/rootfs/initramfs
```

Hình 9. Cấu trúc root filesystem và file initramfs.cpio.gz.

```
drm-cld-pl111 10020000.cld: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #1: ADI AC97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1
Please press Enter to activate this console. Input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000/notherboard-bus@40000000/notherboard-bus@40000000/lofpga07.00000000/10007000.kml/serloi1/input/input2
random: fast init done
```

Hình 10. Hệ thống boot thành công và chạy /sbin/init.

Dòng “Boot completed! Welcome to Embedded Linux Lab 1” xác nhận init script đã được thực thi thành công.

```
/ # mount
rootfs on / type rootfs (rw,size=244916K,nr_inodes=61229)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /tmp type tmpfs (rw,relatime)
/ # ps
PID USER TIME COMMAND
1 root 0:02 init
2 root 0:00 [kthreadd]
3 root 0:00 [rcu_gp]
4 root 0:00 [rcu_par_gp]
5 root 0:00 [kworker/0:0-rcu]
6 root 0:00 [kworker/0:0H]
7 root 0:00 [kworker/u8:0-evt]
8 root 0:00 [rm_percpu_wq]
9 root 0:00 [ksoftirqd/0]
10 root 0:00 [rcu_sched]
11 root 0:00 [migration/0]
12 root 0:00 [cpuhp/0]
13 root 0:00 [cpuhp/1]
14 root 0:00 [migration/1]
15 root 0:00 [ksoftirqd/1]
16 root 0:00 [kworker/1:0-evt]
17 root 0:00 [kworker/1:0H-evt]
18 root 0:00 [cpuhp/2]
19 root 0:00 [migration/2]
20 root 0:00 [ksoftirqd/2]
21 root 0:00 [kworker/2:0]
22 root 0:00 [kworker/2:0H]
23 root 0:00 [cpuhp/3]
24 root 0:00 [kworker/0:1-rcu]
25 root 0:00 [migration/3]
26 root 0:00 [ksoftirqd/3]
27 root 0:00 [kworker/3:0]
28 root 0:00 [kworker/3:0H]
29 root 0:00 [kdevtmpfs]
30 root 0:00 [inet_frag_wq]
31 root 0:00 [kworker/1:1-tpm]
32 root 0:00 [khungtaskd]
33 root 0:00 [oom_reaper]
34 root 0:00 [writeback]
35 root 0:00 [kcompactd0]
36 root 0:00 [kblockd]
37 root 0:00 [ata_sff]
38 root 0:00 [kworker/u8:1-evt]
39 root 0:00 [rpciod]
40 root 0:00 [kworker/u8:0]
41 root 0:00 [korkiod]
42 root 0:00 [kworker/1:1H]
43 root 0:00 [kworker/u8:2]
44 root 0:00 [kswapd0]
45 root 0:00 [nfsiod]
46 root 0:00 [kworker/1:2-evt]
47 root 0:00 [kworker/u8:3-evt]
51 root 0:00 [irq/33-mmc-pl11]
52 root 0:00 [kworker/1:3]
53 root 0:00 [kworker/0:2-evt]
54 root 0:00 [card0-crtcd]
60 root 0:00 ./bin/sh
67 root 0:00 ps
/ # cat /sys/devices/system/cpu/possible
0-3
/ #
```

Hình 11. Kiểm tra filesystem đã mount và các process đang chạy.

EMBEDDED LINUX SYSTEMS — LAB 01 REPORT

```
67 root 0:00 ps
/ # cat /sys/devices/system/cpu/possible
0-3
/ # cat /sys/devices/system/cpu/online
0-1
/ # grep -c "processor" /proc/cpuinfo
2
/ # uname -a
Linux embedded-lab 5.15.0 #3 SMP Tue Sep 29 18:44:24 +07 2026 armv7l GNU/Linux
/ # cat /proc/cpuinfo | grep -E "processor|model|features"
processor
: 0
model name
: ARMv7 Processor rev 0 (v7l)
Features
: half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
processor
: 1
model name
: ARMv7 Processor rev 0 (v7l)
Features
: half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
/ # cat /proc/meminfo | head -5
MemTotal: 508288 kB
MemFree: 492820 kB
MemAvailable: 490800 kB
Buffers: 0 kB
Cached: 1520 kB
/ #
```

Hình 12. Xác minh ARMv7, SMP 2 CPU online và dung lượng RAM.

Kết quả cho thấy `/sys/devices/system/cpu/online` = 0-1 và `/proc/cpuinfo` có 2 processor, tức hai CPU đang online khi QEMU chạy với `-smp 2`. `uname -a` xác nhận `armv7l` và `SMP`; `MemTotal` khoảng 508288 kB, phù hợp với cấu hình 512 MB RAM.

5. Phân tích và trả lời câu hỏi

Câu 1. So sánh arm-linux-gnueabi và arm-linux-gnueabihf

arm-linux-gnueabi sử dụng ABI kiểu soft-float/softfp, trong khi arm-linux-gnueabihf sử dụng hard-float ABI, cho phép truyền và xử lý tham số dấu phẩy động bằng các thanh ghi FPU theo ABI tương ứng. Cortex-A9 thuộc ARMv7-A và có FPU, vì vậy bài lab chọn gnueabihf để tận dụng FPU phần cứng. Nếu dùng gnueabi, mã vẫn có thể nhắm tới ARM nhưng không sử dụng hard-float ABI như cấu hình của bài lab; đặc biệt userspace và thư viện phải thống nhất ABI với nhau để tránh lỗi không tương thích.

Câu 2. Memory map 0x60000000 và ảnh hưởng đến bootcmd

vexpress-a9 có vùng RAM được ánh xạ bắt đầu tại 0x60000000, khác với Versatile PB. Vì vậy U-Boot phải đặt kernel, initramfs và DTB tại các địa chỉ hợp lệ trong vùng RAM này. Trong bài lab: zImage ở 0x60100000, initramfs ở 0x64000000 và DTB ở 0x60000000. Nếu đặt sai địa chỉ, U-Boot/kernel có thể đọc dữ liệu không hợp lệ, ghi đè các vùng nhớ khác hoặc không tìm thấy image, dẫn tới boot thất bại.

Câu 3. Giải thích MIDR 0x410fc090

MIDR (Main ID Register) dùng để nhận diện CPU. Với giá trị 0x410fc090: implementer 0x41 là ARM; part number 0xC09 tương ứng Cortex-A9; các trường variant và revision mô tả biến thể và revision của implementation CPU. Dòng boot log “CPU: ARMv7 Processor [410fc090]” vì vậy phù hợp với target Cortex-A9/ARMv7-A của bài lab.

Câu 4. Vì sao BusyBox nên build static?

Với initramfs tối giản, static linking giúp BusyBox tự chứa phần mã thư viện cần thiết, nên hệ thống có thể chạy mà không cần dynamic loader và các shared library riêng. Điều này làm rootfs đơn giản hơn và tránh lỗi do thiếu thư viện. Nếu build dynamic cho ARM hard-float, rootfs phải chứa đúng dynamic loader và các thư viện ARM tương ứng. Theo đường dẫn được gợi ý trong đề lab trên Ubuntu, tối thiểu có thể cần copy:

- /usr/arm-linux-gnueabi/lib/ld-linux-armhf.so.3
- /usr/arm-linux-gnueabi/lib/libc.so.6
- /usr/arm-linux-gnueabi/lib/libm.so.6

Ngoài ra, tùy binary có thể cần thêm libgcc_s.so.1 hoặc thư viện khác. Có thể dùng arm-linux-gnueabi-gcc -print-file-name=<library> để xác định file thực tế trên máy.

```
arm-linux-gnueabi-gcc --print-sysroot
arm-linux-gnueabi-gcc -print-file-name=libc.so.6
arm-linux-gnueabi-gcc -print-file-name=libgcc_s.so.1
```

Câu 5. SMP với -smp 2

Khi SMP được bật, kernel phải khởi tạo cơ chế đa CPU, đưa các secondary CPU online, thiết lập per-CPU data, scheduler và cơ chế đồng bộ giữa các core. Với -smp 1 chỉ có một CPU hoạt động; với -smp 2, hệ thống có CPU 0 và CPU 1. Trong kết quả thực hành, /sys/devices/system/cpu/online trả về 0-1 và số dòng processor trong /proc/cpuinfo là 2. uname -a cũng chứa SMP. Đây là bằng chứng thực tế cho thấy kernel SMP đang chạy với hai CPU online.

6. Nhận xét và khó khăn gặp phải

Khó khăn	Nguyên nhân	Cách xử lý
Kernel Panic: Unable to mount root fs	Boot kernel nhưng chưa cung cấp initramfs/root filesystem.	Bổ sung -initrd ~/embedded_lab1/output/initramfs.cpio.gz và dùng rdinit=/sbin/init khi boot hoàn chỉnh.
Thoát QEMU bằng Ctrl+A rồi X không hoạt động trong phiên terminal	Tổ hợp escape không được terminal/QEMU nhận như mong đợi.	Mở terminal khác và dùng pkill qemu-system-arm để dừng tiến trình QEMU.
cpu/possible hiển thị 0-3 trong khi chạy -smp 2	possible biểu diễn tập CPU mà platform/kernel có thể hỗ trợ, không nhất thiết là số CPU đang online.	Kiểm tra /sys/devices/system/cpu/online và đếm processor trong /proc/cpuinfo; kết quả thực tế là 0-1 và 2 CPU.

7. Kết luận

Bài lab đã hoàn thành chuỗi khởi động Linux nhúng ARMv7 trên QEMU vexpress-a9. Kernel 5.15 được cấu hình với initramfs, PL011 console và SMP; U-Boot được dùng để kiểm tra và thiết lập bootargs/bootcmd; root filesystem tối giản được đóng gói thành initramfs. Hệ thống cuối cùng boot thành công vào userspace/BusyBox, nhận ARMv7, hai CPU online và khoảng 512 MB RAM.

8. Tài liệu tham khảo

LAB01_EmbeddedLinux.docx — tài liệu hướng dẫn chính của bài lab.

Linux Kernel Documentation — tài liệu tham khảo được liệt kê trong đề.

U-Boot Documentation — tài liệu tham khảo được liệt kê trong đề.

BusyBox Documentation — tài liệu tham khảo được liệt kê trong đề.

QEMU ARM Documentation — tài liệu tham khảo được liệt kê trong đề.